

MÓDULO 7

Diccionarios y Conjuntos

Sesión 7 · 3 horas — objetivo: manejar diccionarios (claves, métodos, anidados) y conjuntos (unión, intersección, diferencia).

¿Qué es un diccionario?

Colección de pares clave-valor. La clave es la etiqueta descriptiva; el valor, el dato asociado.

nombre	:	"Ana López"
edad	:	22
ciudad	:	"La Paz"
activo	:	True
promedio	:	88.5

```
# Crear diccionario
alumno = {
    "nombre": "Ana López",
    "edad": 22,
    "ciudad": "La Paz",
    "activo": True,
    "promedio": 88.5
}

# Acceder
print(alumno["nombre"])
print(alumno.get('tel', 'N/A'))
```



Clave única

No pueden repetirse



Mutable

Agregar, cambiar, eliminar



Ordenado

Python 3.7+ mantiene orden



O(1)

Búsqueda en tiempo constante

Acceso, modificación y métodos

Dos formas de acceder: corchetes `[]` (lanza `KeyError`) y `get()` (retorna `None` o un default).

```
# Acceder
alumno["nombre"]      # "Ana" ✓
alumno["telefono"]    # KeyError ✗
alumno.get("telefono") # None ✓
alumno.get("tel", "N/A") # "N/A" ✓

# Agregar / modificar
alumno["carrera"] = "Ingeniería"
alumno["edad"]    = 23

# Eliminar
del alumno["activo"] # elimina
v = alumno.pop("ciudad") # elim.+retorna
v = alumno.pop("x", None) # sin KeyError

# Verificar existencia
"nombre" in alumno # True
```

`keys()`

Todas las claves

`values()`

Todos los valores

`items()`

Pares (clave, valor)

`update(d2)`

Fusiona otro dict

`setdefault(k,v)`

Agrega si no existe

💡 Usa `get()` siempre que no estés 100% seguro de que la clave existe. Evita `KeyError`.

Recorrer diccionarios

Tres formas de recorrer un diccionario con for — cada una para un propósito diferente.

```
precios = {'Laptop':3500.0, 'Mouse':85.0, 'Teclado':220.0}
```

for k in dict / .keys()

```
for producto in precios:  
    print(producto)  
# 'Laptop'  
# 'Mouse'  
# 'Teclado'
```

.values() — solo valores

```
for precio in precios.values():  
    print(f'Bs. {precio:.2f}')  
# Bs. 3500.00  
# Bs. 85.00
```

.items() — pares ← el más usado

```
for prod, precio in precios.items():  
    print(f'{prod}: Bs. {precio}')  
# Laptop: Bs. 3500.0
```

```
total = sum(precios.values()) # 3805.0 — funciones incorporadas sobre .values()
```

Diccionarios anidados

Un diccionario puede tener otros diccionarios como valores — ideal para datos jerárquicos.

"EMP001"

```
nombre: "Ana López"  
cargo:  "Desarrolladora"  
salario: 4500.0
```

"EMP002"

```
nombre: "Carlos Ramos"  
cargo:  "Diseñador"  
salario: 3800.0
```

```
# Acceder: dict[k1][k2]  
empleados["EMP001"]["nombre"] # 'Ana López'  
empleados["EMP002"]["salario"] # 3800.0
```

```
# Modificar anidado  
empleados["EMP001"]["salario"] = 5000.0
```

```
# Recorrer diccionario anidado  
for id_emp, datos in empleados.items():  
    print(f"{id_emp}: {datos['nombre']:<15} Bs. {datos['salario']:,.2f}")  
# EMP001: Ana López Bs. 5,000.00
```


Conjuntos – sets

Colección desordenada de elementos ÚNICOS. Ideal para eliminar duplicados y operaciones de conjuntos.

Lista con duplicados

```
nums = [1,2,3,2,1,4,3,5,1]  
len(nums) # 9 elementos
```

→ `set()` →

Set – sin duplicados

```
unicos = {1, 2, 3, 4, 5}  
len(unicos) # 5
```



$A \cup B$

Unión – todos los elementos



$A \cap B$

Intersección – en A y en B



$A - B$

Diferencia – en A, no en B



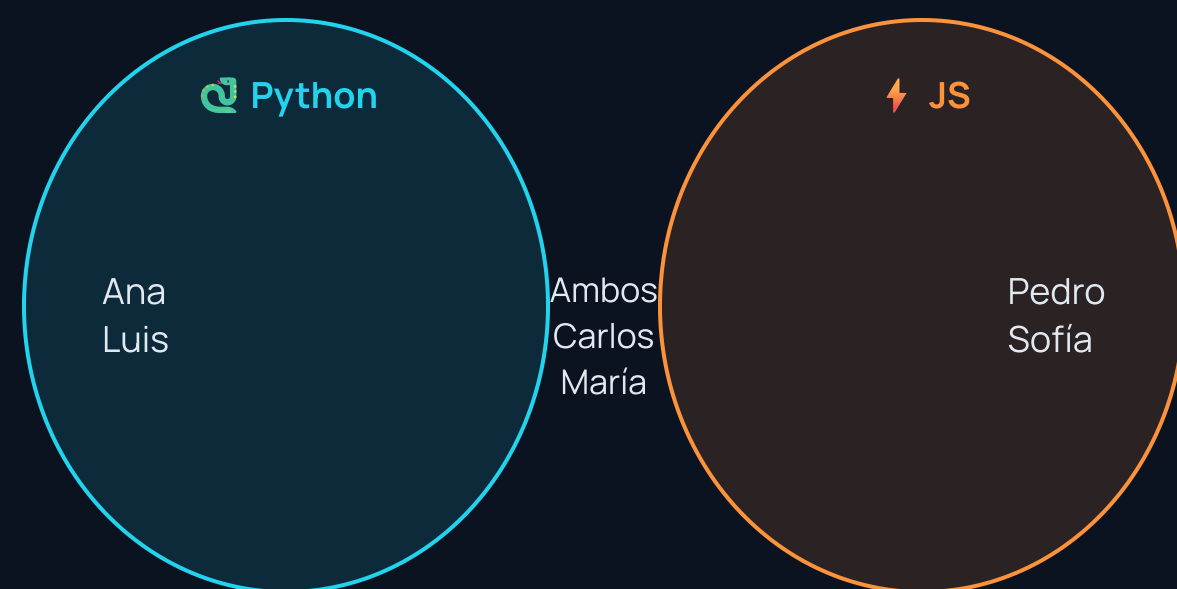
$A \oplus B$

Dif. simétrica – en uno, no ambos

⚠️ `{}` crea un DICCIONARIO vacío. Para un conjunto vacío usa `set()`.

Conjuntos – operaciones en práctica

Caso real: analizar qué desarrolladores dominan Python, JavaScript o ambos.



```
py = {'Ana', 'Carlos', 'María', 'Luis'}
js = {'Carlos', 'Pedro', 'María', 'Sofía'}

# Unión
py | js  # todos
# Intersección
py & js  # ambos
# Diferencia
py - js  # solo py

union = py | js  # {Ana, Carlos, María, Luis, Pedro, Sofía}
ambos = py & js  # {Carlos, María}
solo_py = py - js  # {Ana, Luis}
```

💡 Los conjuntos son más rápidos que las listas para verificar pertenencia ('x' in set vs 'x' in list). $O(1)$ vs $O(n)$.

¿Cuándo usar cada estructura de datos?

La elección correcta es clave para escribir código eficiente y legible.



Lista []

Orden importa, duplicados permitidos

```
notas=[85,92,78,95]
```



Tupla ()

Datos fijos e inmutables

```
coords=(19.04,-65.26)
```



Diccionario {}

Buscar por etiqueta, acceso $O(1)$

```
{'nombre': 'Ana', 'edad': 22}
```



Conjunto set

Elementos únicos, sin orden

```
{'leer', 'escribir'}
```

```
# Combinación real: clientes con sus compras (lista) y su perfil (dict)
```

```
clientes = {'Ana': {'compras': [150, 280, 90],  
                    'categoria': 'VIP'}}
```


Ejercicio integrador — Sistema de inventario

 Combinar diccionarios anidados, conjuntos y listas en un sistema real de tienda

```
inventario = {
    "P001":{"nombre":"Laptop","precio":3500,"stock":5,"cat":"comp"},
    "P002":{"nombre":"Mouse", "precio":85, "stock":20,"cat":"perf"},
    "P003":{"nombre":"Monitor","precio":1200,"stock":3,"cat":"comp"},
}
# Categorías únicas con set
cats = {p['cat'] for p in inventario.values()} # {'comp','perf'}
# Agrupar por categoría con dict
por_cat = {}
for cod,datos in inventario.items():
    c = datos['cat']
    por_cat.setdefault(c, [])
    por_cat[c].append(datos['nombre'])
# Total del inventario
total = sum(d['precio']*d['stock'] for d in inventario.values())
print(f"Valor total: Bs. {total:,.2f}")
```



Dict anidado

inventario



Set

cats — únicas



Dict + Lista

por_cat



List comp.

agrupar productos



`setdefault(k,v)`: crea la clave si no existe; si ya existe, la deja intacta.

✓ LO QUE APRENDISTE HOY

Módulo 7 en resumen



Diccionario

Pares clave-valor, acceso $O(1)$



Claves únicas

Inmutables (str, int, tupla)



Métodos

.keys() .values() .items() .get() .pop()



Anidados

dict[k1][k2] para datos jerárquicos



Conjuntos

Elementos únicos, sin orden



Unión & $\cap$ - $\wedge$

Operaciones matemáticas



Rendimiento

set: $O(1)$ vs list $O(n)$



Elegir

Lista/Tupla/Dict/Set según el problema



Próxima sesión: **Módulo 8 — Proyecto Final** (try/except, archivos, módulos)